

Cognitive Architectures for Robotics

Standalone Localization Benchmark for Humanoid Robot Indoor Operation

Contents

1	Introduction and Research Problem	2
2	State of the Art	2
3	Research Formulation	3
3.1	Objectives	3
3.2	Hypotheses	3
4	Research Methodology	4
5	Implementation and Package Organization	5
6	Cognitive Environmental	5
6.1	Mapping and Database Generation	5
6.2	Reactive Obstacle Avoidance: The Subconscious Layer	6
7	Preliminary Results: Dynamic Localization Benchmarking	6
7.1	Uni-modal Vulnerability and Covariance Explosion	6
7.2	Multi-modal Robustness and Sensor Fusion	7
8	Experimental Scenarios	7
8.1	Scenario 1: Nominal Conditions and Moderate Noise	7
8.2	Scenario 2: Sensory Uncertainty Stress (High Noise)	8
8.3	Scenario 3: Dynamic Obstacles	9
8.4	Scenario 4: Robot Kidnapping and Recovery	9
8.5	Summary	9
9	References	10

1 Introduction and Research Problem

The deployment of mobile robots in highly complex, unstructured, and dynamic environments poses significant algorithmic and architectural challenges. For autonomous systems to operate safely and effectively, they must excel in navigation, real-time mapping, and precise localization. However, current frameworks often fall short when transitioning from controlled laboratory settings to industrial-grade applications.

One of the most critical vulnerabilities lies in standard navigation frameworks, which frequently lack the deterministic lifecycle management and rigorous security protocols required for safe operation around humans and dynamic obstacles. Furthermore, a paradoxical challenge arises in the domain of probabilistic localization. Traditional algorithms, such as Monte Carlo Localization (MCL), rely on particle filters to estimate the robot's pose. Counter-intuitively, when these algorithms are paired with highly accurate, low-noise sensors (such as modern LiDARs), they often fail or experience severe performance degradation. This occurs because the true posterior distribution becomes extremely narrow; if the standard proposal distribution fails to place samples exactly within this narrow region, a "sampling mismatch" occurs, leading to particle depletion and localization failure.

Additionally, long-term autonomy is heavily hindered by computational and memory constraints. Simultaneous Localization and Mapping (SLAM) algorithms must continuously match current sensor data against past observations to detect loop closures. As the mapped environment grows, the processing delays grow unmanageably, eventually violating the real-time constraints necessary for autonomous navigation.

At a geometric level, accurately registering and aligning 3-D environmental shapes without relying on pre-extracted features remains a computationally intensive hurdle. Interestingly, these challenges of continuous error-correction and state estimation are not unique to robotics. In parallel stochastic systems found in nature, such as RNA polymerization, dynamical systems must continuously identify and correct mismatch errors through localized backtracking. Understanding and mathematically modeling these biological stochastic processes can provide profound insights into designing robust error-correction algorithms for robotic systems.

2 State of the Art

A review of the current literature reveals significant developments and remaining gaps in achieving long-term, industrial-grade autonomy. The foundational concepts of this research are built strictly upon the following established methodologies:

Navigation Architectures: While legacy solutions like the original ROS Navigation have been widely adopted, they have failed to keep up with modern trends. The state-of-the-art transition is represented by Navigation2 (Nav2) on ROS2. As highlighted by Macenski et al. [1], modern navigation solutions require Behavior Trees for task orchestration, secure message-passing frameworks suitable

for safety-critical applications, and strict program lifecycle management to operate reliably in dynamic environments.

Geometric Alignment: For the accurate registration of 3-D shapes, the Iterative Closest Point (ICP) algorithm remains the standard. Described by Besl and McKay [2], ICP is a general-purpose, representation-independent method that converges monotonically to the nearest local minimum of a mean-square distance metric. However, achieving global minimization requires an adequate set of initial states, confirming that pure geometric alignment necessitates careful initialization to avoid converging to incorrect local minima.

Probabilistic Localization: Monte Carlo Localization (MCL) has enjoyed great practical success using particle filters for mobile robot localization. However, Thrun, Fox, and Burgard [3] identified a counter-intuitive limitation: highly accurate sensors can actually yield worse results. The true posterior distribution becomes extremely narrow, causing standard sampling steps to fail (particle depletion). Addressing this requires advanced proposal distributions, such as Mixture MCL, to maintain robustness.

SLAM and Memory Bottlenecks: In simultaneous localization and mapping (SLAM), continuous operation over large scales inevitably leads to unbounded computational growth. Labbé and Michaud [4] demonstrate that without explicit constraints, map growth severely impacts real-time performance. Their RTAB-Map approach mitigates this by utilizing appearance-based loop closure detection coupled with strict memory management, which is essential for large-scale and long-term online operation.

Biological Error Correction: Furthermore, complex error-correction mechanisms can be modeled mathematically. Zuo and Chou [5] explored stochastic models of backtracking in RNA polymerization, providing exact analytic solutions for error correction times. These biological parallel stochastic systems inspire the implementation of bounded, localized error-correction in robotic state estimation.

3 Research Formulation

To address the aforementioned gaps, this research formulates a comprehensive cognitive architecture that guarantees real-time performance, algorithmic robustness, and deterministic execution.

3.1 Objectives

The primary objective of this research is to develop robust, modular, and computationally efficient robotic architectures capable of real-time state estimation, dynamic error correction, and secure autonomous navigation over indefinite periods.

3.2 Hypotheses

The research is driven by four core hypotheses:

Task Orchestration: Employing behavior trees combined with the modern ROS2 middleware will provide secure, deterministic, and easily reconfigurable task orchestration compared to traditional finite state machines.

Memory Management: By introducing a cognitive-inspired memory management system that strictly limits the size of the “Working Memory”, SLAM algorithms will be able to process loop closures within fixed, predictable time limits, enabling indefinite online operation.

Localization Robustness: A Mixture MCL algorithm that combines traditional sampling with a dual proposal distribution will successfully eliminate the degradation associated with highly accurate sensors.

3-D Registration: The ICP algorithm will successfully register arbitrary 3-D shapes and point clouds by reliably minimizing the mean-square distance between data sets, providing a dependable baseline for geometric alignment.

4 Research Methodology

To test these hypotheses, the research employs a multi-faceted methodology spanning system architecture, probabilistic sampling, and stochastic modeling:

System Architecture (Navigation2): The research leverages the Navigation2 (Nav2) framework in ROS2. By utilizing a Behavior Tree navigator, complex navigation logic is broken down into modular, reusable nodes that delegate specific tasks (planning, controlling, recovery) to asynchronous servers. This ensures a deterministic and highly fault-tolerant lifecycle.

Mapping Pipeline (RTAB-Map): To solve the SLAM memory bottleneck, the research implements RTAB-Map (Real-Time Appearance-Based Mapping). This system mimics cognitive processes by dividing data into Short-Term, Working, and Long-Term Memory. When the Working Memory reaches a set threshold, older or less crucial nodes are transferred to Long-Term Memory, guaranteeing capped processing times for loop closure detection regardless of the environment’s total size.

Probabilistic Sampling (Mixture MCL): The localization flaw is addressed by creating a Mixture MCL algorithm. This methodology combines standard forward-guessing (based on odometry) with a “backward” dual sampling approach (generating samples directly from the sensor observations). This ensures that even with perfect sensors, particles are always populated in the correct regions, dramatically increasing robustness.

Shape Registration and Stochastic Modeling: The ICP algorithm is applied to handle the alignment of 3-D environmental data optimally. Furthermore, to understand the fundamental limits of error-correction, the research formulates discrete stochastic state models. By applying Laplace transforms to these models, explicit error-correction and first-passage times can be mathematically calculated, drawing inspiration from biological backtracking processes like RNA transcription.

5 Implementation and Package Organization

To systematically evaluate the proposed cognitive architecture and localization hypotheses, the software implementation is divided into modular ROS2 packages within a containerized workspace. This separation ensures reproducibility and logical isolation between simulation assets, robot kinematics, and benchmarking algorithms.

- **aws-robomaker-small-house-world:** This package encapsulates the 3-D simulation environment. It contains the structural meshes, textures, and Gazebo World configurations (.sdf files) representing the indoor domestic setting used for all experimental trials.
- **g1_description:** This package holds the physical and kinematic definitions of the humanoid robot. It includes the Unified Robot Description Format (URDF) models (both 2-D and 3-D variations) alongside the visual and collision meshes necessary for accurate physics simulation in Gazebo.
- **gazebo_g1:** Acting as the integration layer, this package bridges the robot description with the simulation world. It houses the primary launch files required to spawn the robot, configure the Navigation2 stack, and store the foundational map data (e.g., the RTAB-Map .db database and 2-D occupancy grids).
- **g1_loc_benchmark:** The core benchmarking suite developed for this research. It contains the orchestration logic for the various stress tests:
 - **Launch Configurations:** Scripts like `benchmark_amcl.launch.py` to instantiate specific experimental scenarios with custom sensor noise or actor spawning.
 - **Dynamic Modules:** Python nodes responsible for spawning dynamic human obstacles (`dynamic_obstacles.py`), simulating anthropomorphic walking (`g1_controller.py`), and managing autonomous exploration (`auto_explorer.py`).
 - **Evaluation Suite:** The `benchmark_evaluator.py` node, which synchronously records real-time odometry drift by comparing estimated poses against the Gazebo ground truth, and `dashboard.py`, a Plotly-based web application for post-experiment data visualization.

6 Cognitive Environmental

6.1 Mapping and Database Generation

Before the robot can effectively localize, a foundational spatial representation of the environment must be constructed. This was achieved using the RTAB-Map SLAM framework operating in a multimodal configuration (processing both RGB-D and 3-D LiDAR data). This process heavily mimics the cognitive function of spatial encoding. During the mapping phase, the robot explored the environment, synthesizing visual features and geometric point clouds into a highly dense global graph. Once exploration concluded, this graph was serialized into a persistent .db database, which acts as the robot’s immutable “Long-Term Memory” during pure localization tasks.

However, a critical limitation in this cognitive-inspired mapping approach stems from hardware and computational constraints. A raw, unfiltered 3-D point cloud of an entire house contains an overwhelming volume of data points, which would cause the navigation's local costmap to violate real-time processing deadlines. To mitigate this, a strict geometric filtering strategy was applied during database generation. Specifically, the system was configured to ignore objects below a certain height threshold (e.g., `Grid/MaxGroundHeight` set to 0.02m and `Grid/MaxObstacleHeight` set to 1.5m). While this filtering is mandatory to prevent the robot from misinterpreting minor ground irregularities (such as carpets or cables) as impassable walls, it inherently caps the system's absolute accuracy. The robot becomes cognitively "blind" to very low-profile physical impediments, resulting in an environment representation that is highly optimized for navigation but intentionally lacks 100% geometric fidelity.

6.2 Reactive Obstacle Avoidance: The Subconscious Layer

In a cognitive architecture, while global path planning acts as the deliberate, "conscious" reasoning layer responsible for finding the shortest route from A to B, the local obstacle avoidance functions as the reactive, "subconscious" layer.

As the robot executes the global plan, it continuously relies on its local costmap, which is updated in real-time by the LiDAR sensors. To ensure safety, obstacles are virtually expanded using an `inflation_radius` corresponding to the robot's physical footprint. When an unmapped, dynamic obstacle (such as a walking human) suddenly intersects the planned trajectory, the system utilizes a Dynamic Window Approach (DWA) controller. This controller instantaneously simulates multiple short-term velocity trajectories (Twist commands) and scores them based on collision probability, path adherence, and goal progression. The trajectory with the highest score is executed, allowing the robot to smoothly and unconsciously swerve around dynamic threats without the computationally heavy burden of recalculating the entire global route.

7 Preliminary Results: Dynamic Localization Benchmarking

Initial benchmarking experiments were conducted utilizing the described ROS2 simulated environment, featuring scripted dynamic actors to induce stress conditions (sensor occlusion and unmodeled kinematic collisions) on the navigation stack. The comparative results highlight a critical divergence in robustness between uni-modal and multi-modal localization strategies.

7.1 Uni-modal Vulnerability and Covariance Explosion

When relying exclusively on 2-D LiDAR data via the AMCL algorithm, the system exhibited severe vulnerability in dynamic scenarios. As dynamic actors crossed the robot's path, the LiDAR rays reflected off the moving obstacles rather than the mapped static environment, causing significant localized sensor occlusion.

This vulnerability catastrophically escalated during physical collisions. When the robot was struck and physically displaced, the system suffered an irreconcilable kinematic discrepancy. The internal motion model (odometry) recorded zero displacement since no active motor commands were issued, while the external sensor model perceived an abrupt shift. Lacking reliable static geometric anchors due to the concurrent occlusion, the AMCL algorithm failed to correct this discrepancy. Consequently, the particle filter experienced a “covariance explosion”: the particle cloud diverged rapidly, resulting in an irrecoverable loss of global localization and a cascading failure of the navigation stack.

7.2 Multi-modal Robustness and Sensor Fusion

Conversely, the multi-modal configuration—combining 2-D LiDAR with 3-D RGB-D visual odometry via RTAB-Map—demonstrated substantial resilience under the same stress conditions. During identical dynamic occlusion and collision events, the visual sensors continued to track elevated 3-D environmental features (e.g., upper walls and distant structural keypoints) that the low-profile dynamic actors could not occlude.

This continuous stream of reliable, elevated visual keypoints successfully anchored the robot’s pose estimation, effectively overriding the kinematic discrepancy caused by the collision. The sensor fusion architecture prevented the covariance explosion, enabling the system to quickly re-converge to the correct pose and dynamically update the local costmap to navigate around the actors. These results empirically validate the hypothesis that multi-modal sensor fusion is strictly necessary to maintain robust, long-term autonomy in environments populated by dynamic obstacles.

8 Experimental Scenarios

Experimental trials were conducted across four distinct scenarios, designed to test the limits of the system’s pose estimation under conditions of increasing stress.

Note on Methodology and Hardware Limitations: It is important to specify that the following stress benchmarks (Scenarios 1-4) were conducted exclusively using the uni-modal AMCL (2D LiDAR) configuration. Real-time dynamic tracking using the multi-modal (ICP and RGB-D) approach was computationally infeasible during live simulation due to the extreme heaviness of the 3-D .db database generated by RTAB-Map, which caused severe hardware bottlenecks. Furthermore, a critical limitation emerged from the geometric filtering applied to this database: because point-cloud data below a certain height threshold was intentionally ignored to save memory, the robot became entirely “blind” to obstacles lower than that limit. Since these low-profile obstacles were not detected or registered in the database, the robot could not perceive or avoid them during navigation.

8.1 Scenario 1: Nominal Conditions and Moderate Noise

Under optimal visibility conditions and standard Gaussian noise (LiDAR stddev of 0.02m), the navigation system maintained a consistent pose estimation, completing paths toward the goal without

significant deviations. The translation error remained consistently below 0.05m, validating the system's baseline behavior.

8.2 Scenario 2: Sensory Uncertainty Stress (High Noise)

Testing the system with elevated LiDAR noise (standard deviation of 0.05 m) and drastically reducing the AMCL filter particles (max_particles: 100, min_particles: 50) resulted in noticeable performance degradation. The time required to reach the navigation goal increased significantly. With a poorer probabilistic representation, the filter exhibited high uncertainty during rotations, leading to less fluid trajectories and frequent corrective adjustments to re-converge with the map.

To observe the system's absolute failure point, the noise standard deviation was further increased to 0.5 m alongside a severe reduction in the particle count. As demonstrated in extreme-noise, this extreme configuration triggers a catastrophic loss of localization. The excessive simulated noise produces a profoundly distorted and hyper-dense point cloud that entirely obscures the real environmental features. Consequently, the AMCL algorithm is starved of reliable data, loses all spatial context, and the robot becomes completely unable to navigate.

Traiettorie: Extreme noise - localization_results.csv

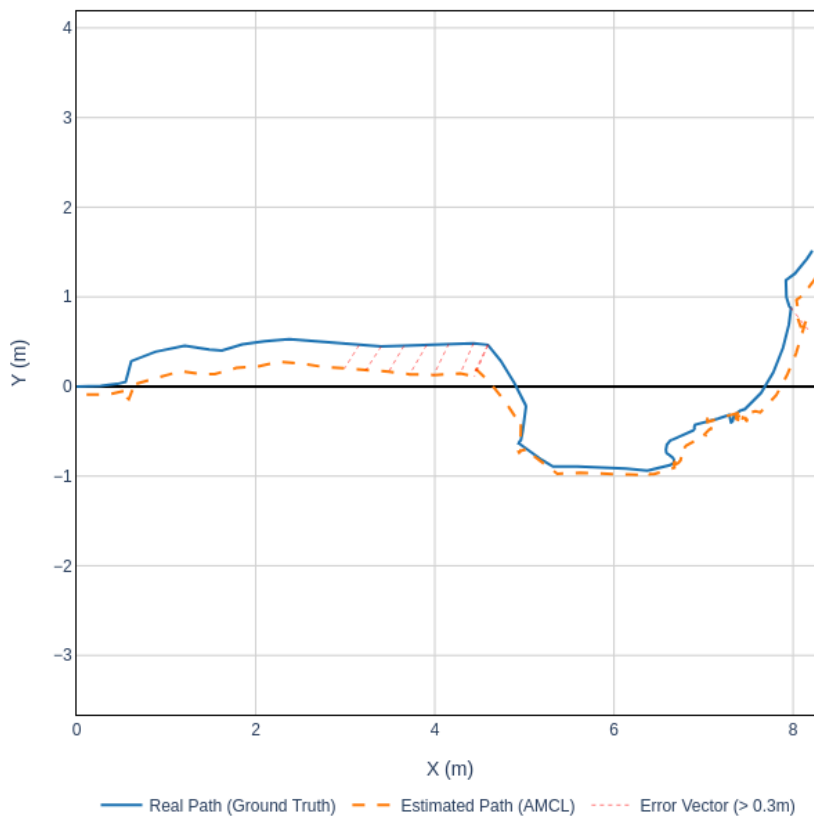


Figure 1: Effect of extreme sensory noise and particle starvation on robot localization.

8.3 Scenario 3: Dynamic Obstacles

By introducing 6 actors with heterogeneous dynamics (variable speeds on X and Y axes) and optimizing the LiDAR update rate (30Hz) and local_costmap frequency (20Hz), the robot demonstrated superior reactive capabilities. Reducing the cost_scaling_factor to 3.0 and increasing the inflation_radius to 0.8m allowed the robot to deviate from obstacles preventatively. Note on limitations: Despite the optimization, collisions occurred when dynamic obstacles entered the robot's "blind spot" radius. The navigation responded with emergency maneuvers (spin, backup), which proved ineffective when the obstacle struck the robot laterally or posteriorly—areas not covered by the front-facing 2D LiDAR sensor.

8.4 Scenario 4: Robot Kidnapping and Recovery

The kidnapping test was performed via manual teleportation of the robot in Gazebo (variable x, y). At the moment of the abduction, the AMCL algorithm initially attempted to maintain the previous pose, leading to an immediate error explosion (exceeding 10 meters).

Recovery Procedure: To validate robustness, a "forced exploration" phase was implemented via teleop_twist_keyboard. By rotating the robot on its axis and moving it forward for a few meters, the filter triggered recovery via recovery_alpha_fast.

Results: The system successfully reinitialized the particle cloud, re-converging to the correct pose after geometric scanning of the new environment. The error trend shows a vertical spike at the time of the event, followed by an asymptotic decay curve as the robot regains localization, confirming that the system is capable of self-correction if supported by active exploratory movement after displacement.

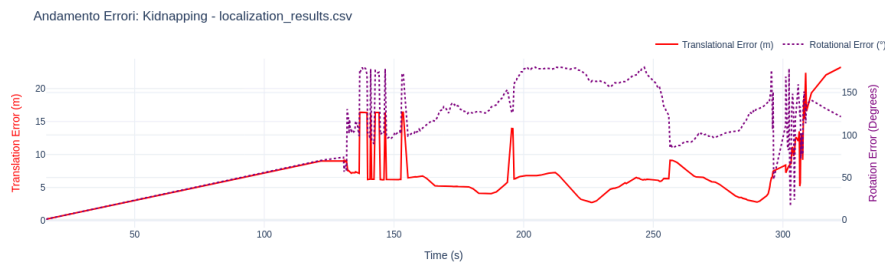


Figure 2: Error trend during a kidnapping event, demonstrating the spike and subsequent asymptotic recovery.

8.5 Summary

In conclusion, the evaluation of the AMCL localization framework reveals a system that is highly reliable under nominal conditions but exhibits distinct vulnerabilities under specific environmental stresses.

As illustrated in Figure 3, the comparative analysis of mean translational and rotational errors across all tested scenarios highlights these operational limits. The baseline scenario (Nominal Conditions)

maintains near-zero error margins, proving the fundamental accuracy of the baseline Nav2 stack. However, the introduction of disturbances proportionally increases system uncertainty.

The most significant sustained errors occur during the Sensory Uncertainty (High Noise) scenario, demonstrating that the AMCL filter is highly susceptible to sensor degradation; once the geometric structure of the point cloud is lost, the algorithm cannot maintain spatial context. Conversely, the Kidnapping scenario produces the largest absolute error spike, yet the system proves its resilience by successfully re-converging to the ground truth once active exploratory commands are issued. Finally, the Dynamic Obstacles test highlights a critical hardware limitation rather than a purely algorithmic one: while the local costmap efficiently routes the robot around frontal dynamic threats, the limited field-of-view of the 2D LiDAR leaves the robot vulnerable to lateral and posterior collisions.

Ultimately, this benchmark indicates that while the core AMCL algorithm is robust and capable of self-recovery, ensuring safe deployment in complex, highly dynamic real-world environments would require a multimodal sensor fusion approach to mitigate physical blind spots and noise susceptibility.

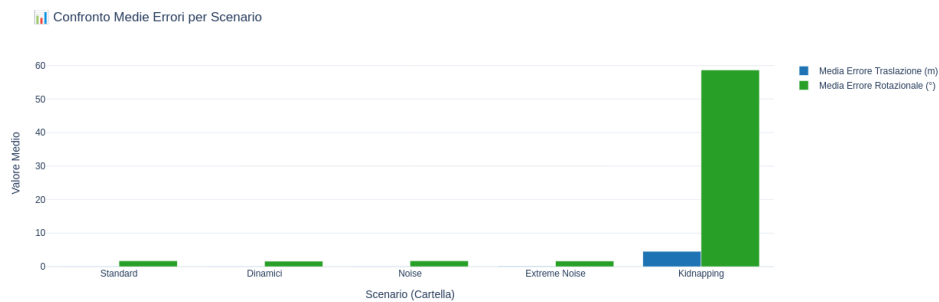


Figure 3: Comparative analysis of mean translational and rotational errors across the evaluated scenarios.

9 References

- [1] Macenski, S., Martín, F., White, R., & Ginés Clavero, J. (2020). **The Marathon 2: A Navigation System**. arXiv:2003.00368v2.
- [2] Besl, P. J., & McKay, N. D. (1992). **A Method for Registration of 3-D Shapes**. IEEE Transactions on Pattern Analysis and Machine Intelligence, 14(2), 239-256.
- [3] Thrun, S., Fox, D., & Burgard, W. (2000). **Monte Carlo Localization With Mixture Proposal Distribution**.
- [4] Labbé, M., & Michaud, F. (2018). **RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation**. Journal of Field Robotics.
- [5] Zuo, X., & Chou, T. (2021). **Density- and elongation speed-dependent error correction in RNA polymerization**. arXiv:2103.05151v1.